JAVA

- Java is high level, object oriented programming language developed by Sun Microsystems of USA in 1991.
- Java is used in a wide variety of computing platforms from embedded devices and mobile phones on the low end, to enterprise servers and supercomputers on the high end.

JAVA

- Features of Java are as follows:

1. Compiled and Interpreted
2. Platform Independent and portable
3. Object- oriented
4. Robust and secure
5. Distributed
6. Familiar, simple and small
7. Multithreaded and Interactive
8. High performance
9. Dynamic and Extensible

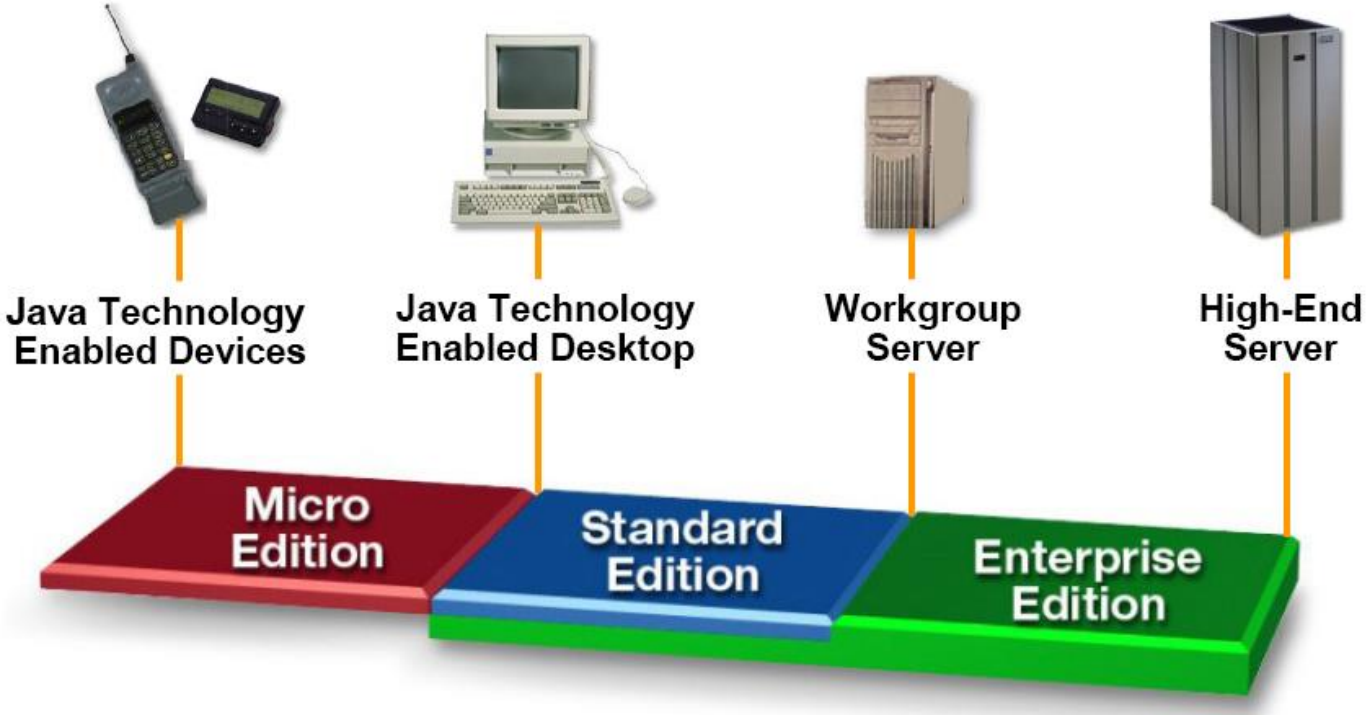
Java platform

- Java Platform
  - Set of software products and specifications from Sun Microsystems/Oracle Corporation that provides a way to develop applications and deploy them in a cross-platform environment
- Java platform main components:
  - Java language compiler
  - Java Virtual Machine
  - Class libraries
- Java Programing Language
  - Language core of the Java Platform which is the main one used to produced code for the Java Platform

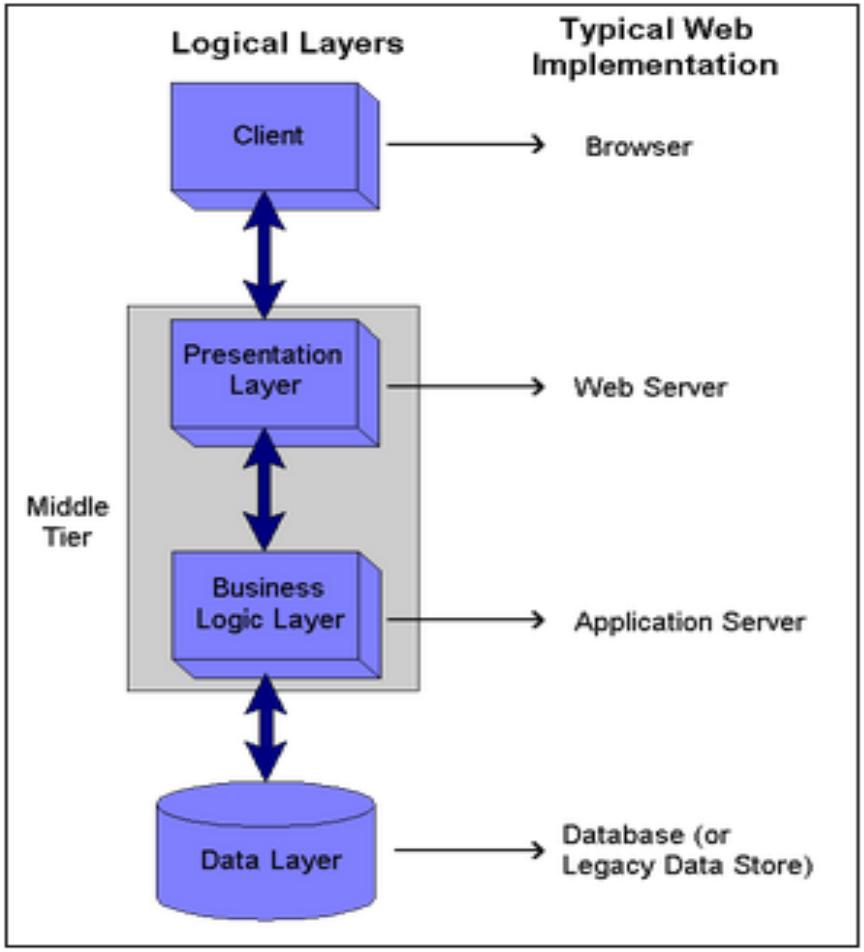
Java Platform Editions

1. **Java SE (Standard Edition)-** is a widely used platform for development and deployment of portable applications for desktop and server environments.
2. **Java EE (Enterprise Edition)-** platform provides an API and runtime environment for developing and running enterprise software, including network and web services, and other large-scale, multi-tiered, scalable, reliable, and secure network applications.
3. **Java ME (Micro Edition)-** is a Java platform designed for embedded systems (mobile devices are one kind of such systems). Target devices range from industrial controls to mobile phones (especially feature phones) and set-top boxes.

Java Platform Editions



General Web Architecture



General Web Architecture

- The client layer of a web application is implemented as a web browser running on the user's client machine. Its job in a web-based application is to display data and let the user enter/update data.
- Broadly, one of two general approaches is used for building the client layer:
  - a "dumb" HTML-only client: with this approach, virtually all the intelligence is placed in the middle tier. When the user submits the webpages, all the validation is done in the middle tier and any errors are posted back to the client as a new page.
  - a semi-intelligent HTML/JavaScript client: with this approach some intelligence is included in the webpage which runs on the client. For example, the client will do some basic validations. The client may also include some dynamic HTML. Note: client intelligence can be built using other browser scripting languages

General Web Architecture

- The Presentation Layer
  - The presentation layer generates webpages and it includes dynamic content in the webpage. The dynamic content typically originates from a database. The other major job of the presentation layer is to "decode" the webpages coming back from the client.
  - Most modern websites presentation layers are either built using: the Microsoft solution using Active Server Pages (ASP) OR the Java solution using some combination of Servlets and Java Server Pages (JSP).

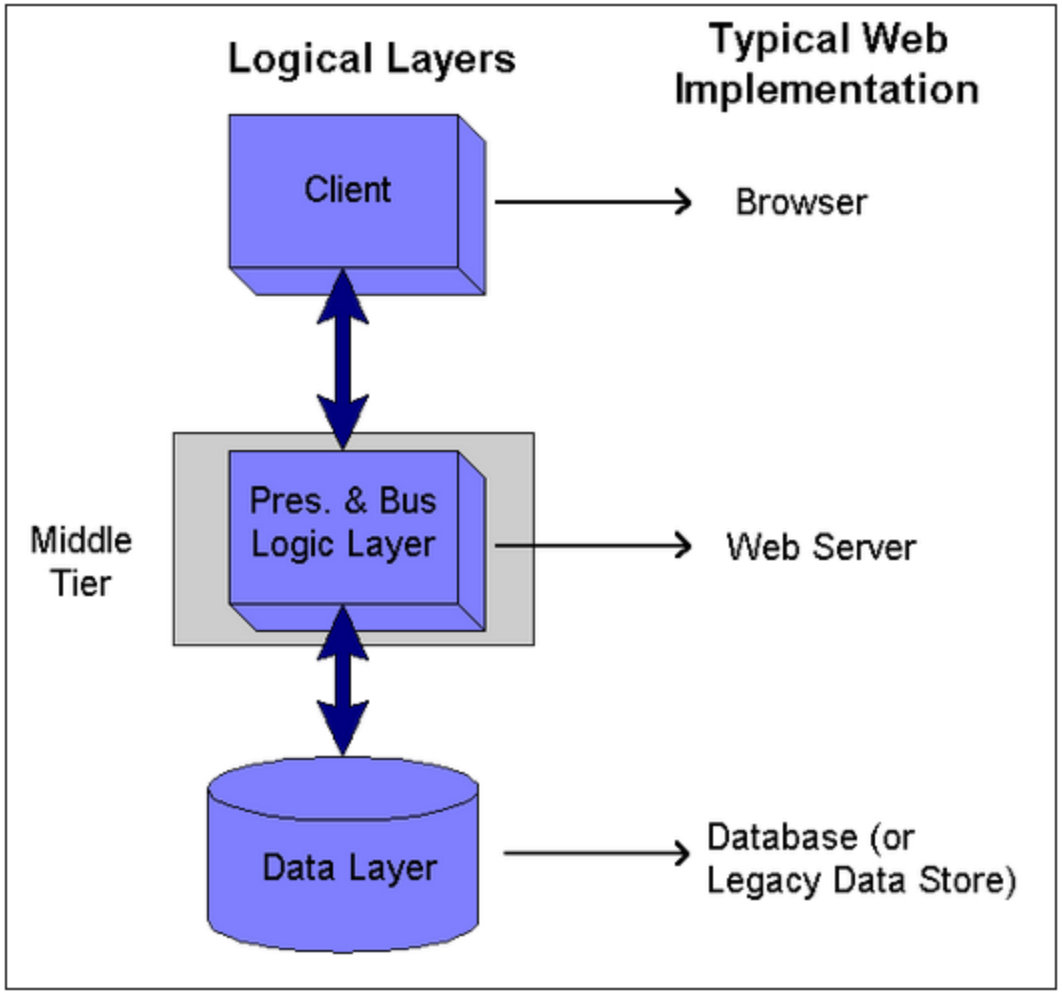
General Web Architecture

- The Business Logic Layer: The bulk of the application logic is written in the business logic layer. Business logic includes:
  - performing all required calculations and validations
  - managing workflow (including keeping track of session data)
  - managing all data access for the presentation tier

General Web Architecture

- The Data
  - The data layer is responsible for managing the data. In the simple case, a data layer may simply be a modern relational database. However, it may include data access procedures to other data sources like hierarchical databases, legacy flat files, etc. The job of the data layer is to provide the business logic layer with required data when needed and to store data when requested.

Simplified Architecture for Smaller Web Applications



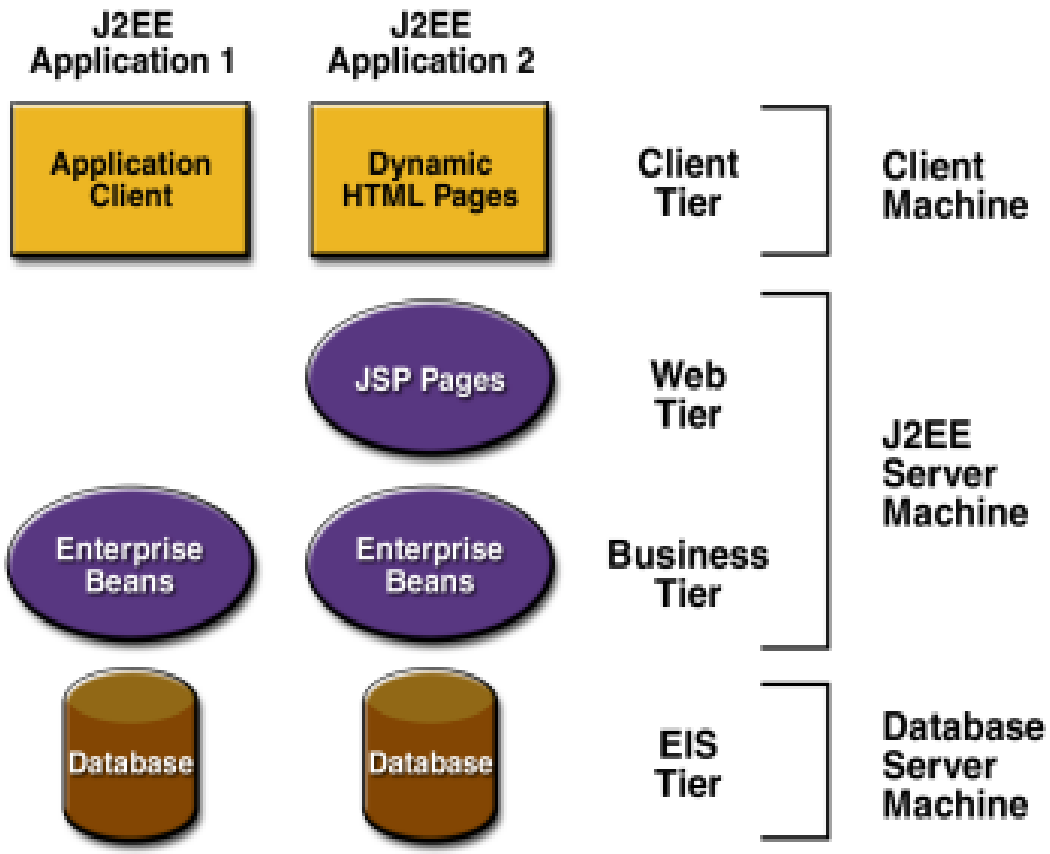
Introduction to J2EE

- **Java 2 Platform, Enterprise Edition (J2EE)** provides a component-based approach to design, development, assembly, and deployment of enterprise applications.
- **Web services** are Web-based enterprise applications that use open-XML-based standards and transport protocols to exchange data with calling clients. The J2EE platform provides the XML APIs and tools you need to quickly design, develop, test, and deploy Web services and clients that fully interoperate with other Web services and clients running on Java-based or non-Java-based platforms.

J2EE Distributed Multitier Applications

- The J2EE platform uses a multitier distributed application model for enterprise applications.
- Application logic is divided into components according to function, and the various application components that make up a J2EE application are installed on different machines depending on the tier in the multitier J2EE environment to which the application component belongs.

J2EE Distributed Multitier Applications



J2EE Distributed Multitier Applications

- Client-tier components run on the client machine.
- Web-tier components run on the J2EE server.
- Business-tier components run on the J2EE server.
- Enterprise information system (EIS)-tier software runs on the EIS server.

J2EE Components

- J2EE applications are made up of components. A J2EE component is a self-contained functional software unit that is assembled into a J2EE application with its related classes and files and that communicates with other components.
- The J2EE specification defines the following J2EE components:
  - Application clients and applets are components that run on the client.
  - Java Servlet and Java Server Pages (JSP) technology components are Web components that run on the server.
  - Enterprise JavaBeans (EJB) components (enterprise beans) are business components that run on the server.

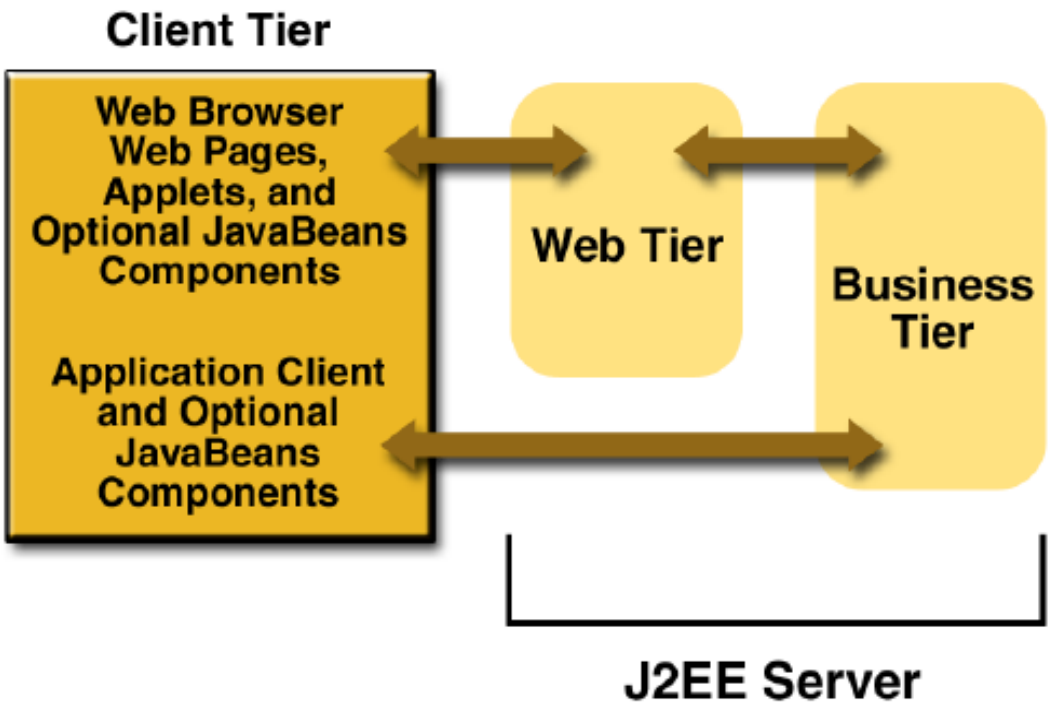
J2EE Components

- A **J2EE client** can be a Web client or an application client.
- A **Web client** consists of two parts: dynamic Web pages containing various types of markup language (HTML, XML, and so on), which are generated by Web components running in the Web tier, and a Web browser, which renders the pages received from the server.
- A Web page received from the Web tier can include an embedded applet. An **applet** is a small client application written in the Java programming language that executes in the Java virtual machine installed in the Web browser.

J2EE Components

- **J2EE application client** runs on a client machine and provides a way for users to handle tasks that require a richer user interface than can be provided by a markup language. It typically has a graphical user interface (GUI) created from Swing or Abstract Window Toolkit (AWT) APIs, but a command-line interface is certainly possible.

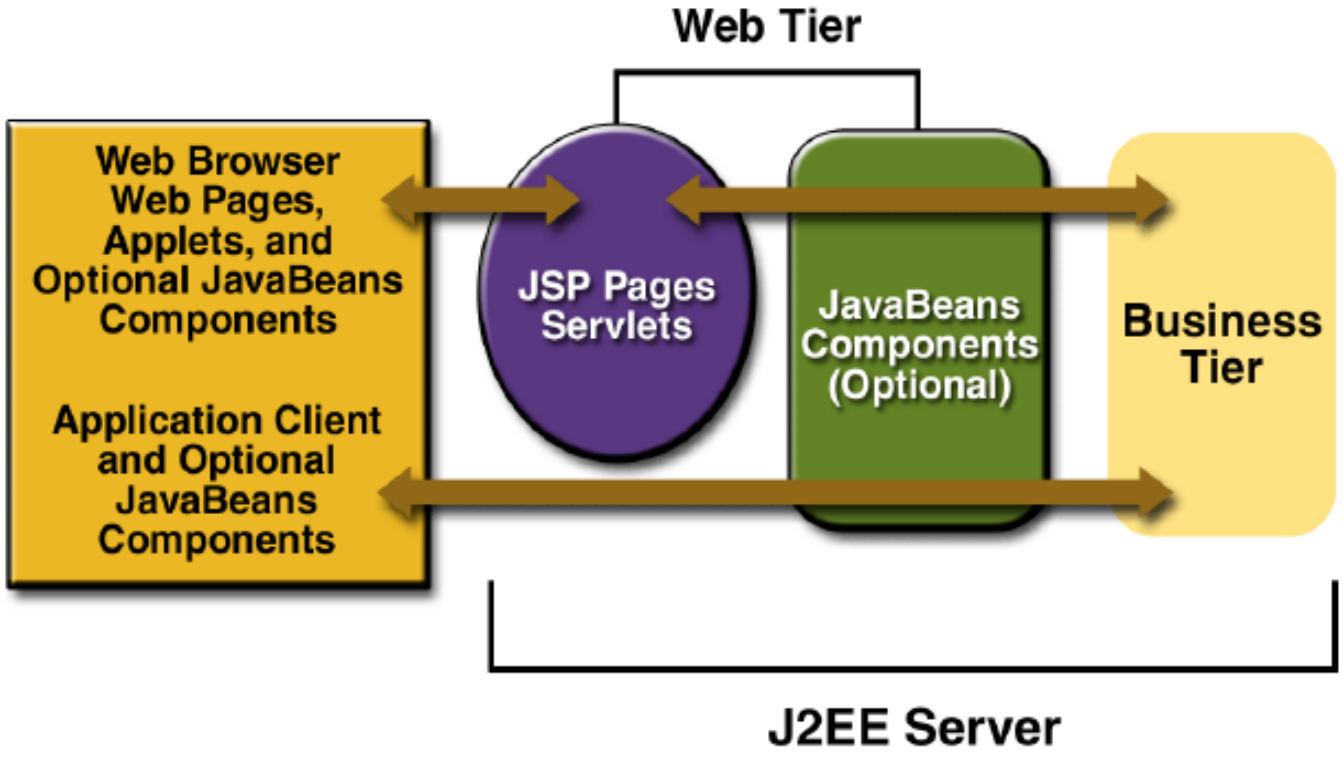
J2EE Server Communications



Web Components

- J2EE Web components can be either servlets or JSP pages.
- *Servlets are Java programming* language classes that dynamically process requests and construct responses.
- *JSP pages are text-based documents that execute as servlets but allow* a more natural approach to creating static content.

Web Components

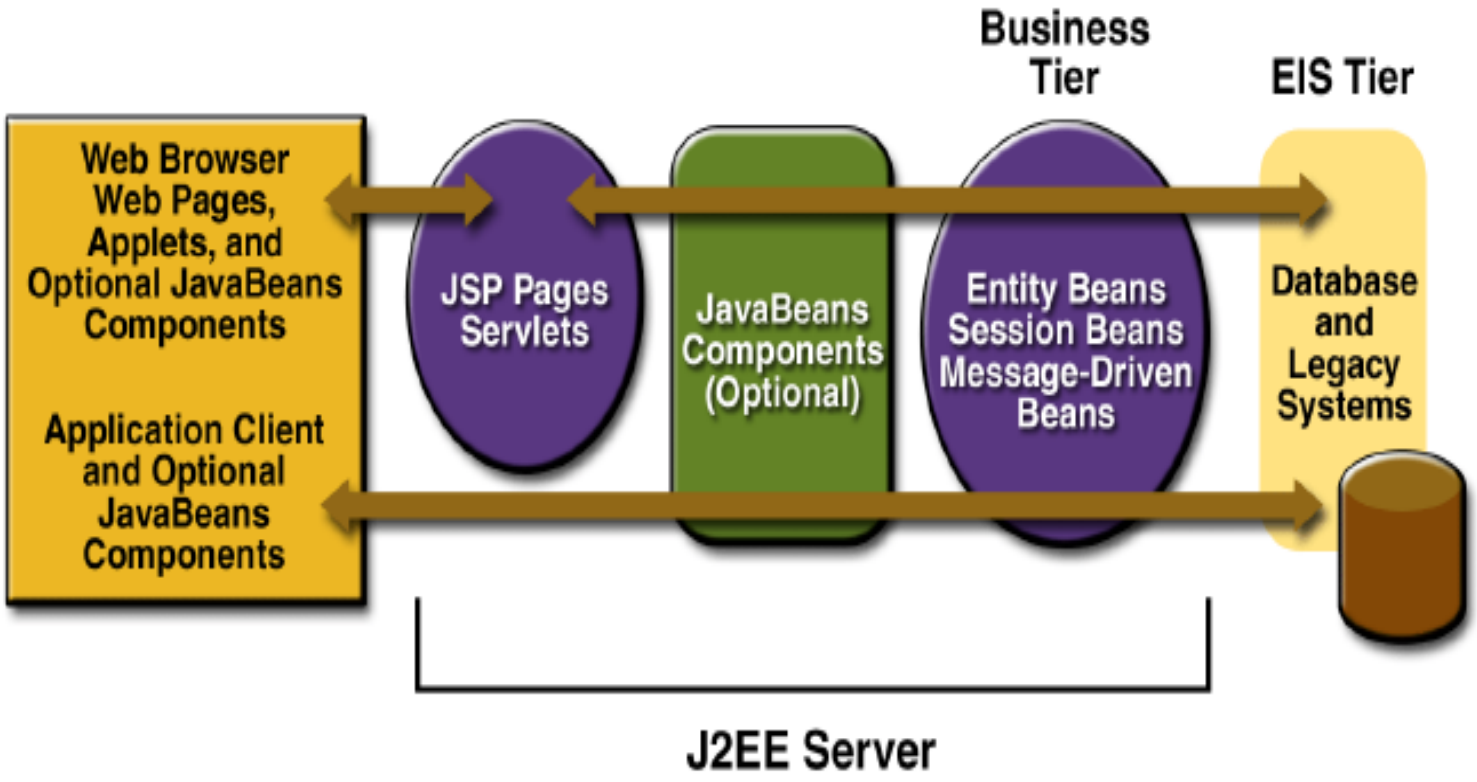


Business Components

- Business code, which is logic that solves or meets the needs of a particular business domain such as banking, retail, or finance, is handled by enterprise beans running in the business
- There are three kinds of enterprise beans:

1. A *session bean represents a transient conversation with a client.* When the client finishes executing, the session bean and its data are gone.
2. In contrast, an *entity bean represents persistent data stored in one row of a database* table. If the client terminates or if the server shuts down, the underlying services ensure that the entity bean data is saved.
3. A *message-driven bean combines features of a session bean and a Java Message* Service (“JMS”) message listener, allowing a business component to receive JMS messages asynchronously.

Business Components



J2EE Container

- *Containers are the interface between a component and the low-level platform specific functionality that supports the component*.
- Before a Web, enterprise bean, or application client component can be executed, it must be assembled into a J2EE application and deployed into its container.

J2EE Container

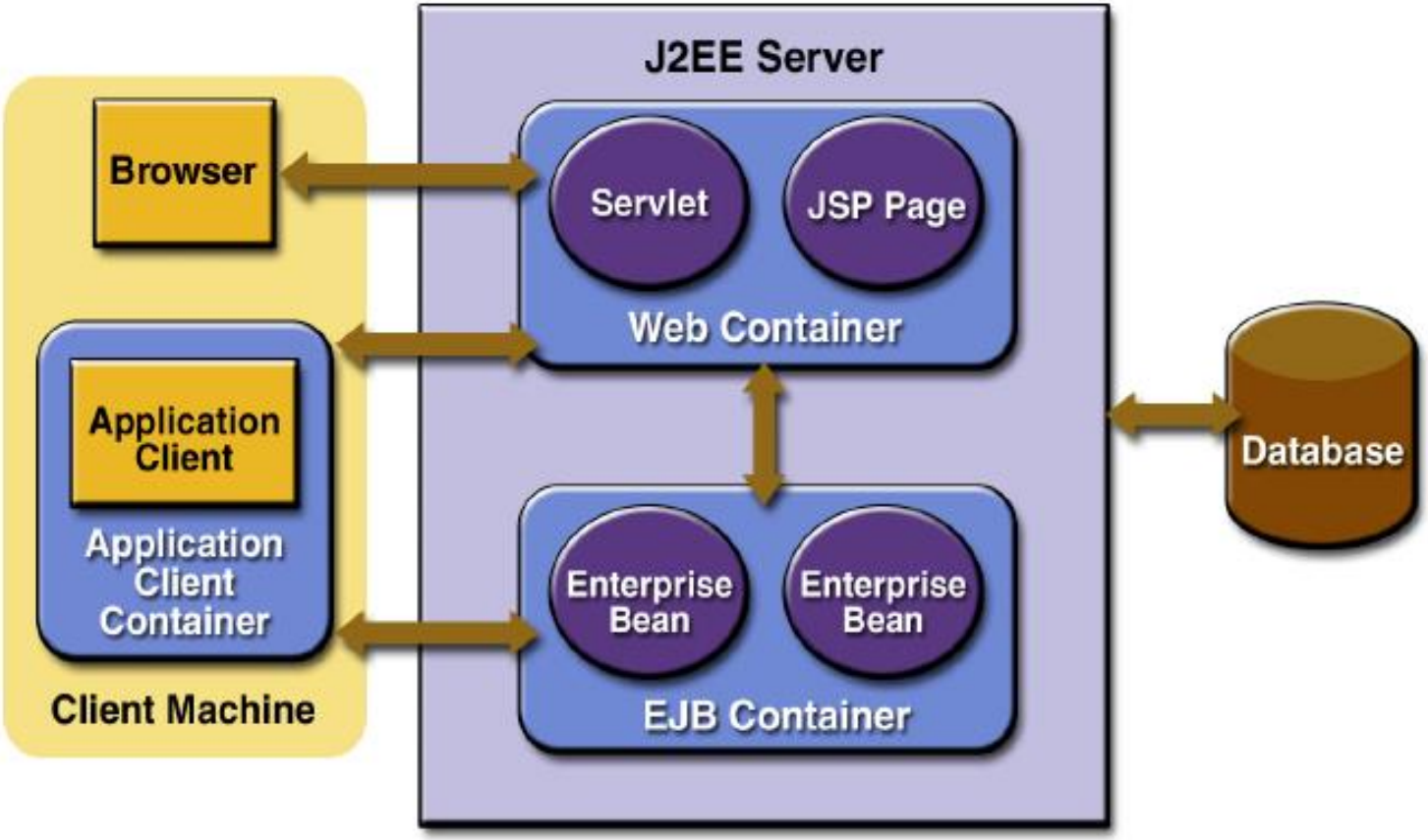
- The container is a runtime environment that controls the enterprise beans and provides them with important system-level services. Since you don't have to develop these services yourself, you are free to concentrate on the business methods in the enterprise beans.

J2EE Container Service

- The container provides the following services:

  1. The J2EE **security** model lets you configure a Web component or enterprise bean so that system resources are accessed only by authorized users.
  2. The J2EE **transaction** model lets you specify relationships among methods that make up a single transaction so that all methods in one transaction are treated as a single unit.
  3. **JNDI lookup** services provide a unified interface to multiple naming and directory services in the enterprise so that application components can access naming and directory services.
  4. The J2EE **remote connectivity** model manages low-level communications between clients and enterprise beans. After an enterprise bean is created, a client invokes methods on it as if it were in the same virtual machine.

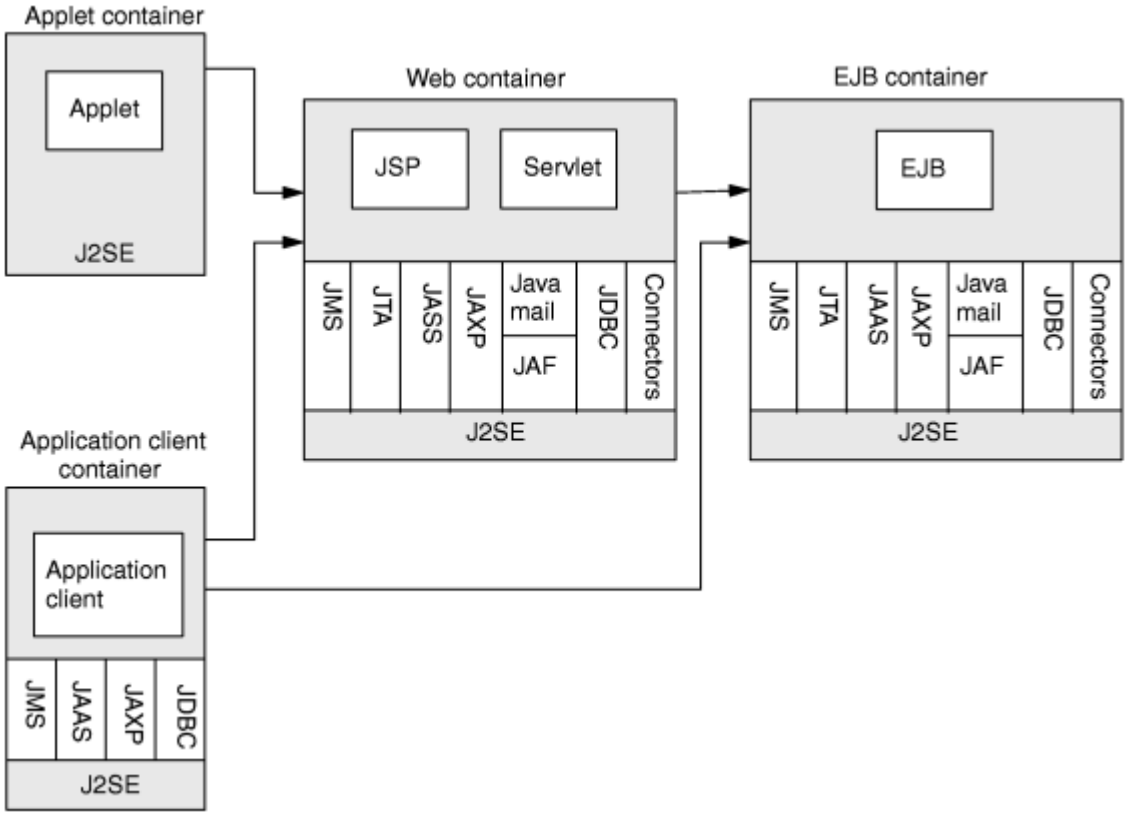
Container Types



Container Types

- **J2EE server** The runtime portion of a J2EE product. A J2EE server provides EJB and Web containers.
- **Enterprise JavaBeans (EJB) container** Manages the execution of enterprise beans for J2EE applications. Enterprise beans and their container run on the J2EE server.
- **Web container** Manages the execution of JSP page and servlet components for J2EE applications. Web components and their container run on the J2EE server.
- **Application client container** Manages the execution of application client components. Application clients and their container run on the client.
- **Applet container** Manages the execution of applets. Consists of a Web browser and Java Plugin running on the client together.

J2EE Architecture



J2EE APIs

- The **JDBC API** lets you invoke SQL commands from Java programming language methods.
- The **Java Message Service (JMS)** is a messaging standard that allows J2EE application components to create, send, receive, and read messages.
- The **Java Naming and Directory Interface (JNDI)** provides naming and directory functionality. It provides applications with methods for performing standard directory operations, such as associating attributes with objects and searching for objects using their attributes. Using JNDI, a J2EE application can store and retrieve any type of named Java object.
- The **Java Transaction API (JTA)** provides a standard interface for demarcating transactions. The J2EE architecture provides a default auto commit to handle transaction commits and rollbacks.

J2EE APIs

- J2EE applications can use the **JavaMail** API to send e-mail notifications. The JavaMail API has two parts: an application-level interface used by the application components to send mail, and a service provider interface. The J2EE platform includes JavaMail with a service provider that allows application components to send Internet mail.
- The **JavaBeans Activation Framework (JAF)** is included because JavaMail uses it. It provides standard services to determine the type of an arbitrary piece of data, encapsulate access to it, discover the operations available on it, and create the appropriate JavaBeans component to perform those operations.

J2EE APIs

- The Java API for **XML Processing (JAXP)** supports the processing of XML documents using Document Object Model (DOM), Simple API for XML Parsing (SAX), and XML Stylesheet Language Transformation (XSLT). JAXP enables applications to parse and transform XML documents independent of a particular XML processing implementation.
- The Java API for **XML Registries (JAXR)** lets you access business and general purpose registries over the Web.

J2EE APIs

- The Java API for **XML-based RPC** (JAX-RPC) uses the SOAP standard and HTTP so client programs can make XML-based remote procedure calls (RPCs) over the Internet.
- The **SOAP with Attachments API for Java** (SAAJ) is a low-level API upon which JAX-RPC depends. It enables the production and consumption of messages that conform to the SOAP 1.1 specification and SOAP with Attachments note. Most developers will not use the SAAJ API, but will use the higher-level JAX-RPC API instead.

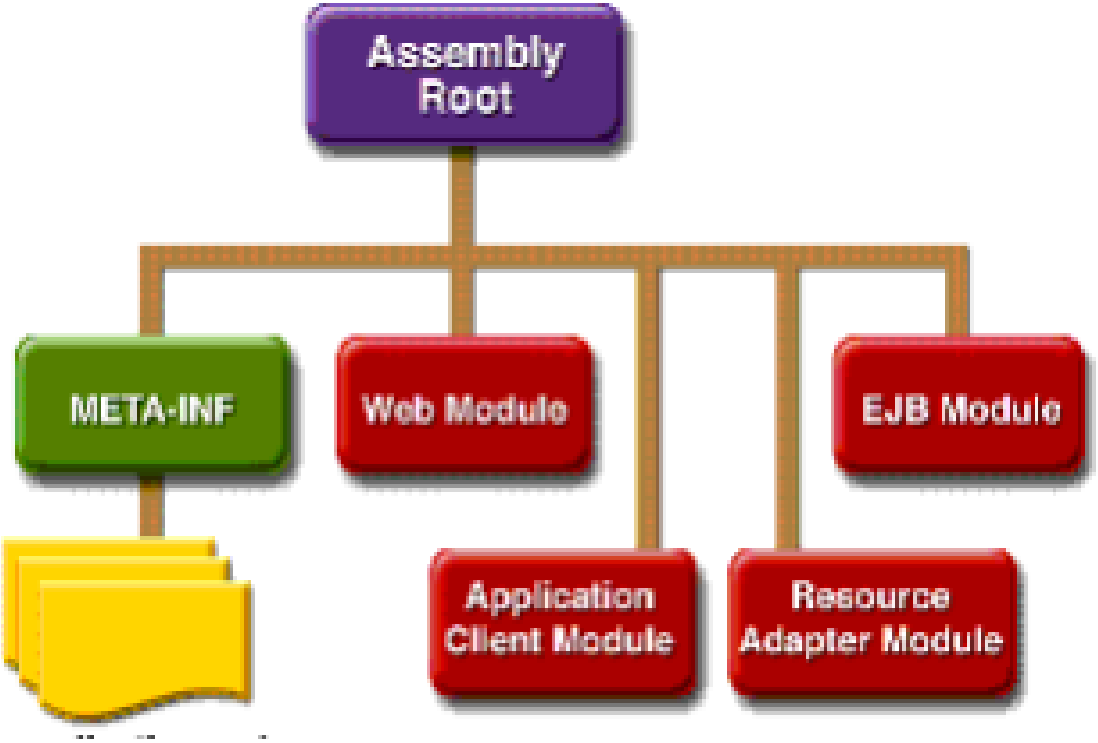
J2EE Pakages

- A J2EE application is delivered in an Enterprise Archive (EAR) file. An EAR file is a standard Java Archive (JAR) file with an **.ear** extension.
- The EAR file contains J2EE modules. Using EAR files and modules makes it possible to assemble a number of different J2EE applications using some of the same components.
- No extra coding is needed; it is just a matter of assembling various J2EE modules into J2EE EAR files.

J2EE Pakages

- An EAR file contains **J2EE modules and deployment descriptors.** A deployment descriptor is an XML document with an.xml extension that describes the deployment settings of an application, a module, or a component.
- Because deployment descriptor information is declarative, it can be changed without modifying the bean source code. At run time, the J2EE server reads the deployment descriptor and acts upon the component accordingly.

Four types of J2EE modules



Four types of J2EE modules

- **EJB modules,** contain class files for enterprise beans and an EJB deployment descriptor. EJB modules are packaged as JAR files with a .jar extension.
- **Web modules,** contain servlet class files, JSP files, supporting class files, GIF and HTML files, and a web application deployment descriptor. Web modules are packaged as JAR files with a .war (web archive) extension.
- **Application client modules,** which contain class files and an application client deployment descriptor. Application client modules are packaged as JAR files with a .jar extension.
- **Resource adapter modules,** which contain all Java interfaces, classes, native libraries, and other documentation, along with the resource adapter deployment descriptor. Together, these implement the Connector architecture for a particular EIS. Resource adapter modules are packaged as JAR files with an .rar (resource adapter archive) extension.

Development Roles

- Reusable modules make it possible to divide the application development and deployment process into distinct roles so that different people or companies can perform different parts of the process.
- The first two roles involve purchasing and installing the J2EE product and tools.
- Once software is purchased and installed, J2EE components can be developed by application component providers, assembled by application assemblers, and deployed by application deployers.

Development Roles

- In the application component development phase, an enterprise bean software developer delivers EJB JAR files.
- In the application assembly role, another developer combines these EJB JAR files into a J2EE application and saves it in an EAR file.
- In the application deployment role, a system administrator at the customer site uses the EAR file to install the J2EE application into a J2EE server.

Web Applications

- A Web application consists of Web components, static resource files such as images, and helper classes and libraries. The Web container provides many supporting services that enhance the capabilities of Web components and make them easier to develop.

Web Application Life cycle

- The process for creating, deploying, and executing a Web application can be summarized as follows:
  1. Develop the web component code.
  2. Develop the web application deployment descriptor.
  3. Compile the web application components and helper classes referenced by the components.
  4. Optionally package the application into a deployable unit.
  5. Deploy the application into a web container.
  6. Access a URL that references the web application.

References

- http://docs.oracle.com/javaee/1.3/tutorial/doc/J2EETutorial.pdf